The products you already have in the client code for Agtron's shop are objects with `name` properties, so a similar `find` function will work. But you do not yet have an array of all the products created. Where should the array of product references go? How about in the `Shop constructor`?

```
class Shop
  constructor: (data) ->
  products = []
  for own name, info of data
    products.push new Product(name, info)
```

But that assumes that an instance of the `Shop` class creates *every* instance of `Product`. Awesome, until somebody `new`s a `Product` outside of the shop and it's never found. Given that `find` is *about* products, why not put it on the `Product` class?

```
Product.find('Pixelmator-X21')
# [ Camera { name="Pixelmator-X21", info={...} } ]
```

The `Product` class can have a property like an object because it *is* an object.

Class methods

Any class *is* an object and has properties just like any other object. To specify a class method named `find`, just assign a function as the `find` property directly on the class:

```
Product.find = (query) ->
  (product for product in products when product.name is query)
```

For that to work you need somewhere to define the `product` variable. You don't want an evil global variable; you want a variable that lives with the `Product` class. You want a class variable.

Class variables

All variables are function-scoped so you're surprised to see what happens when a new variable is assigned in the body of a class declaration: